05. Hermes의 기억법

Hermes Agent가 대화를 거듭하면서 사용자와 환경을 기억하는 방법을 살펴봅니다. 단일 장치가 아니라 용도가 다른 네 가지 장치가 협력하며, 각각의 역할과 한계를 이해하면 에이전트를 훨씬 효과적으로 활용할 수 있습니다.

이 장에서 다루는 내용

- 개요 — 네 가지 기억 장치의 역할과 관계
- 자체 메모리 — MEMORY.md와 USER.md
- 외부 메모리 백엔드 — Honcho를 비롯한 8개 프로바이더
- 세션 검색 — 과거 대화의 전문 검색
- 컨텍스트 압축 — 긴 대화의 자동 요약

05-1. 개요

Hermes Agent는 네 가지 장치를 조합하여 정보를 기억합니다. 각각은 Hermes 내부에서 독립된 시스템으로 구현되어 있으며, 저장 위치·지속 범위·작동 방식이 모두 다릅니다.

- 네 가지 기억 장치
  - 자체 메모리 — "항상 보이는 메모장"
  - 외부 메모리 백엔드 — "학습하는 보조 기억"
  - 세션 검색 — "과거 대화 검색 엔진"
  - 컨텍스트 압축 — "대화 분량 관리"
- 장치 간 관계

네 가지 기억 장치

장치	저장 위치	지속 범위	작동 방식
자체 메모리	`~/.hermes/memories/`	영구 (세션 간)	매 턴 시스템 프롬프트에 자동 주입
외부 메모리 백엔드	외부 서비스 (Honcho 등)	영구 (세션 간)	매 턴 자동 주입 + 도구 호출
세션 검색	`~/.hermes/state.db`	영구 (세션 간)	에이전트가 도구로 검색
컨텍스트 압축	메모리 내 (휘발)	현재 세션 한정	토큰 임계값 도달 시 자동 실행

자체 메모리 — "항상 보이는 메모장"

MEMORY.md(환경·규칙)와 USER.md(사용자 프로필)로 구성됩니다. 용량이 작지만(합계 약 3,500자) 매 턴 시스템 프롬프트에 포함되므로 에이전트가 항상 참조할 수 있습니다.

외부 메모리 백엔드 — "학습하는 보조 기억"

Honcho, Mem0 등 8개 서비스 중 하나를 연결하면, 대화에서 사용자의 선호와 패턴을 자동으로 추출·축적합니다. 자체 메모리의 용량 제약 바깥에서 작동하며, 자체 메모리를 대체하지 않고 보강합니다.

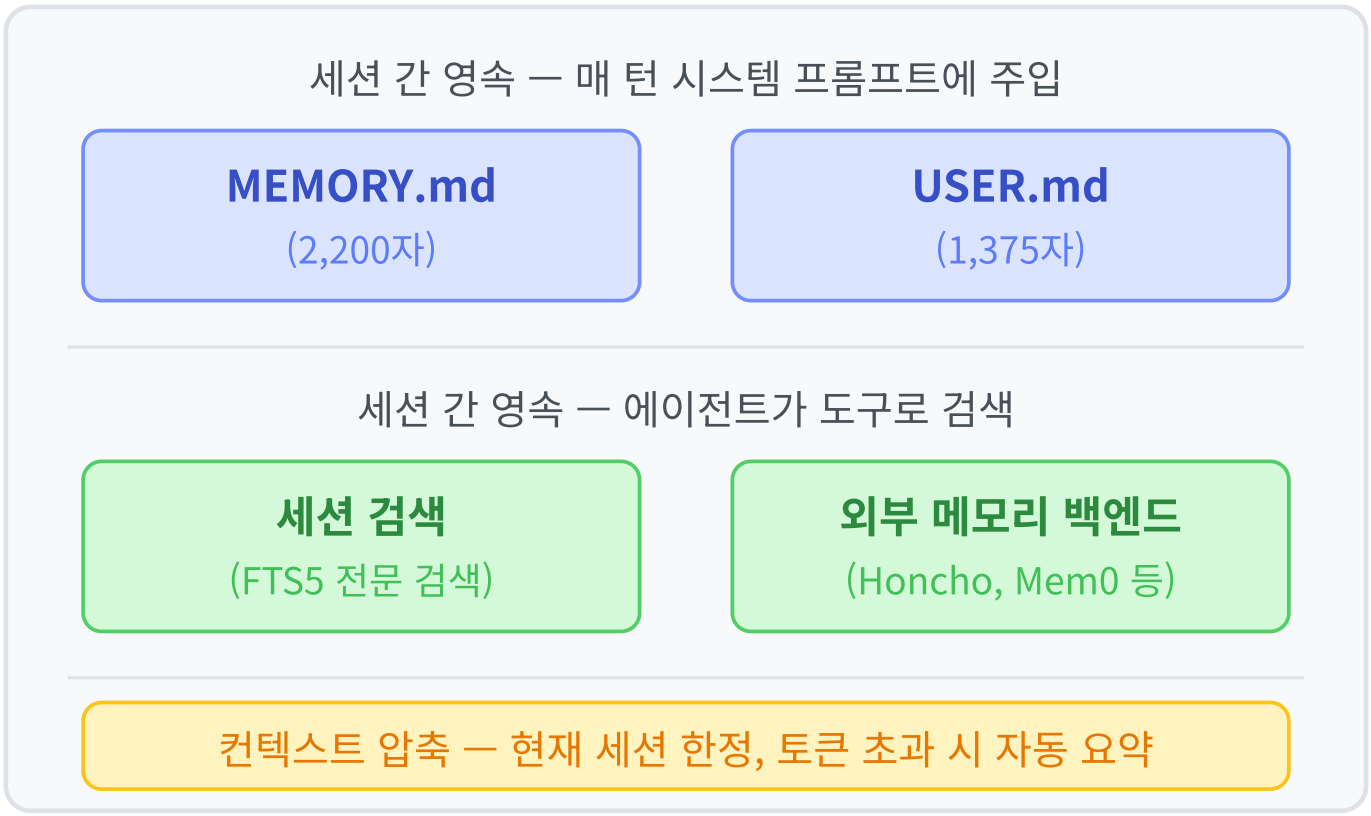
세션 검색 — "과거 대화 검색 엔진"

모든 대화는 SQLite 데이터베이스에 저장됩니다. 에이전트에게 과거 대화를 검색해 달라고 요청하면 전문 검색으로 찾아줍니다. 자체 메모리에 저장하지 못한 정보도 과거 대화에서 찾을 수 있습니다.

컨텍스트 압축 — "대화 분량 관리"

대화가 길어져 LLM의 컨텍스트 윈도우가 가득 차면, 오래된 부분을 자동으로 요약하여 공간을 확보합니다. 다른 세 장치와 달리 현재 세션에서만 작동하며, 세션이 끝나면 요약도 사라집니다.

장치 간 관계



네 장치는 독립적으로 작동합니다. 자체 메모리에 저장한 내용이 세션 검색에 영향을 주거나, 컨텍스트 압축이 메모리 파일을 수정하는 일은 없습니다.

Hermes 내부에서 "메모리"로 분류되는 것은 **자체 메모리와 외부 메모리 백엔드**뿐입니다. 세션 검색과 컨텍스트 압축은 각각 별도의 도구와 파이프라인으로 구현되어 있으며, 시스템 프롬프트에서도 독립된 안내 블록으로 분리되어 있습니다. 이 장에서는 "에이전트가 정보를 유지하는 방법"이라는 사용자 관점에서 네 가지를 함께 다룹니다.

각 장치의 상세는 이어지는 절에서 다루며, 내부 구현은 12장 「아키텍처와 고급 기능」에서 살펴봅니다.

> 💡 ***스킬은 7장에서***: *반복 작업을 자동으로 절차화하는 스킬(Skills)도 넓은 의미에서 에이전트의 기억입니다. 스킬 시스템은 7장에서 별도로 다룹니다.*

05-2. 자체 메모리

Hermes Agent의 자체 메모리는 `~/.hermes/memories/` 디렉터리에 두 개의 파일로 저장됩니다.

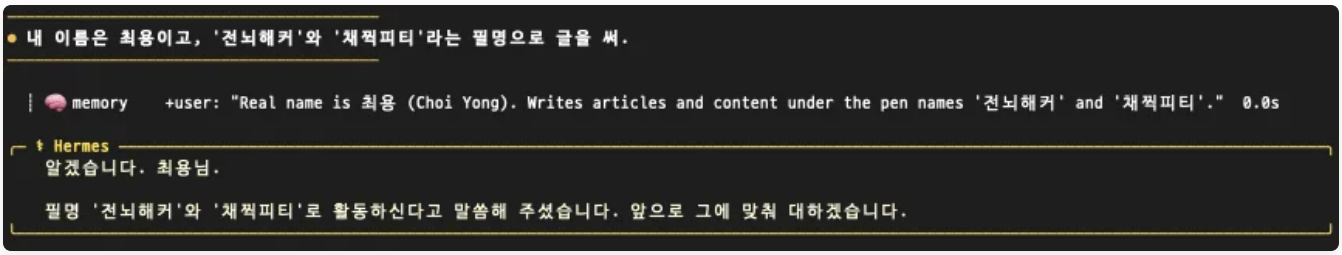
- 메모리에 정보 저장하기
- 용량이 가득 차면
- 직접 관리하기
  - 파일 직접 편집
  - 한도 늘리기
  - 외부 메모리 백엔드 활용

파일	용도	용량
MEMORY.md	환경 정보, 프로젝트 규칙, 교훈, 워크어라운드	최대 2,200자
USER.md	사용자 이름, 역할, 선호, 커뮤니케이션 스타일	최대 1,375자

에이전트는 이 두 파일의 내용을 항상 참조하고 있으므로, 한 번 저장된 정보는 이후 모든 대화에서 활용됩니다.

메모리에 정보 저장하기

사용자 정보를 알려주면 에이전트가 USER.md에 저장합니다.



이어서 홈페이지와 GitHub 아이디를 알려주면 두 번째 항목이 추가됩니다. 파일 안에서 항목은 `§` 기호로 구분됩니다.

```
Real name is 최용 (Choi Yong). Writes articles and content under the pen names '전뇌해커' and
'채찍피티'.
§
Personal website is ychoi.kr. GitHub username is ychoi-kr.
```

한국어로 말해도 영어로 저장될 수 있습니다. 저장 형식은 에이전트가 스스로 결정하며, 사용하는 모델에 따라 달라집니다.

환경이나 도구에 관한 정보는 MEMORY.md에 들어갑니다. 어느 파일에 넣을지는 에이전트가 내용을 보고 판단합니다.

"기억해줘"라고 명시적으로 말하지 않아도, 대화 중에 드러난 선호나 환경 정보를 에이전트가 스스로 저장하는 경우도 있습니다. 10턴 정도 대화하면 백그라운드에서 대화 내용을 검토하여 기억할 만한 정보를 자동으로 추가합니다.

용량이 가득 차면

두 파일의 용량 합계는 약 3,500자로, 의도적으로 작게 설계되어 있습니다. 장기간 사용하면 한도에 도달하는데, 이때 몇 가지 문제가 생깁니다.

- **새 정보가 거부됩니다**: 한도를 넘는 추가는 에러로 처리됩니다.
- **자동 정리가 없습니다**: 오래된 항목을 자동으로 밀어내거나 압축하는 기능이 없습니다. 먼저 쌓인 내용이 공간을 점유하면, 이후 대화에서 알게 된 중요한 정보는 저장되지 못합니다.
- **정리를 시켜도 실패할 수 있습니다**: "메모리를 정리해줘"라고 요청하면 에이전트가 삭제를 시도하지만, 실제로는 삭제되지 않는 경우가 있습니다. 에이전트가 실패를 인식하지 못하고 "정리했습니다"라고 보고할 수도 있습니다.

> ⚠️ ***정리 후에는 직접 확인**: `MEMORY.md` 를 열어 실제 내용을 확인하는 것이 안전합니다.*

직접 관리하기

장기 사용 시에는 파일을 직접 편집하는 것도 좋은 방법입니다.

파일 직접 편집

`MEMORY.md` 와 `USER.md` 를 텍스트 편집기로 열어 필요 없는 항목을 삭제하거나, 관련 항목을 합치면 공간을 확보할 수 있습니다.

> 💡 ***편집 후에는 `/new` 로 새 세션 시작**: 파일을 직접 수정한 경우, 현재 세션에는 반영되지 않습니다. 대화 중에 에이전트가 저장한 내용은 대화 맥락에 남아 있으므로 바로 활용되지만, 편집기로 고친 내용은 새 세션을 시작해야 적용됩니다.*

한도 늘리기

```
# ~/.hermes/config.yaml
memory_char_limit: 4400    # 기본값: 2200
user_char_limit: 2750      # 기본값: 1375
```

한도를 늘리면 더 많은 정보를 저장할 수 있지만, 매 턴 토큰 비용이 증가합니다.

외부 메모리 백엔드 활용

용량 자체가 부족하다면 다음 절의 외부 메모리 백엔드를 함께 쓸 수 있습니다. 핵심 정보는 MEMORY.md에 두고 나머지는 외부 백엔드에 맡길 수 있습니다.

05-3. 외부 메모리 백엔드

MEMORY.md와 USER.md는 매 턴 프롬프트에 포함되므로 용량이 작게 설계되어 있습니다. 외부 메모리 백엔드는 이 용량 제약 바깥에서 사용자에 대한 관찰을 무제한으로 축적하고, 필요할 때만 관련 정보를 꺼내 쓰는 확장 경로입니다.

- 공통 작동 방식
- 메모리 백엔드 비교
- Honcho와 Mem0, 무엇을 고를까

공통 작동 방식

어떤 백엔드를 선택하든, 기본적인 작동 흐름은 동일합니다.

```
대화 → ① 메시지 전송 → ② 사용자 정보 추출·저장 → ③ 다음 턴에 컨텍스트 주입
       (Hermes → 백엔드)   (백엔드 내부 처리)        (백엔드 → Hermes)
```

1단계는 백그라운드로 처리되므로 대화 흐름이 끊기지 않습니다. 2단계에서 백엔드가 LLM을 사용하여 대화에서 사용자에 대한 사실·선호·패턴을 추출합니다. 3단계에서 축적된 정보 중 현재 맥락에 관련된 부분만 골라 시스템 프롬프트에 주입합니다.

모든 외부 백엔드는 자체 메모리(MEMORY.md, USER.md)를 **대체하지 않고 보강**합니다. 자체 메모리가 먼저 로드된 후 외부 백엔드가 추가로 로드되며, 동시에 하나의 백엔드만 활성화할 수 있습니다.

> 💡 ***백엔드 선택은 신중하게:*** *메모리 백엔드 사이에 데이터 마이그레이션 기능은 없습니다. 한 백엔드에 축적한 관찰과 사용자 모델을 다른 백엔드로 옮길 수 없으므로, 처음에 어떤 백엔드를 쓸지 결정하는 것이 중요합니다.*

메모리 백엔드 비교

Hermes는 8개의 메모리 백엔드를 플러그인으로 지원합니다.

백엔드	특징	호스팅
Honcho	변증법적 사용자 모델링, 피어 카드(사용자 프로필 카드), 의미 검색, 추론 깊이 조절	클라우드 / 셀프 호스팅
Hindsight	지식 그래프, 엔티티 해소, 다중 검색 전략	클라우드 / 로컬
RetainDB	파일 업로드·인제스트, 변증법적 종합, 에이전트 셀프 모델	클라우드
OpenViking	ByteDance의 컨텍스트 DB, 파일 계층 구조, 리소스 인제스트	셀프 호스팅
Supermemory	의미 검색, 프로필 리콜, 대화 인제스트	클라우드
Mem0	서버 측 LLM 팩트 추출, 자동 중복 제거	클라우드
ByteRover	CLI 기반, 계층적 컨텍스트 트리, 로컬 우선	로컬 / 클라우드 동기화
Holographic	구조화된 팩트 저장, 엔티티 해소, 신뢰 점수	로컬

백엔드마다 제공하는 도구의 종류와 수가 다릅니다. 적게는 2개(Holographic)에서 많게는 10개(RetainDB)까지 있으며, 프로필 조회·의미 검색·결론 저장 같은 기능은 대부분의 백엔드에 공통입니다.

이 가운데 기능이 가장 풍부한 **Honcho**와 설정이 가장 간단한 **Mem0**를 다음 두 절에서 자세히 다룹니다. 두 백엔드는 같은 외부 슬롯을 두고 경쟁하는 대표적인 선택지입니다.

Honcho와 Mem0, 무엇을 고를까

두 백엔드는 기억하는 대상이 다릅니다.

- **Mem0** — "사실을 정확히 기억한다." 설정이 단순하고 회수가 빠릅니다. 프로젝트 세부 사항·선호·반복 지시처럼 구체적인 사실 중심 작업, 또는 빠르게 시작해 보려는 경우에 적합합니다.
- **Honcho** — "사용자를 이해한다." 사고방식·커뮤니케이션 스타일·작업 패턴을 변증법적으로 모델링합니다. 장기 비서, 글쓰기·기획 같은 창의·전략 작업, 여러 프로필·에이전트를 운영하는 경우에 적합합니다.

확신이 서지 않으면 설정이 간단한 Mem0로 시작해 보는 편이 부담이 적습니다. 다만 앞서 짚었듯 두 백엔드 사이에 데이터 마이그레이션은 없으므로, 한쪽에 관찰을 충분히 쌓은 뒤 바꾸면 그동안의 기록은 옮겨지지 않습니다.

05-3-1. Honcho — 변증법적 사용자 모델

외부 메모리 백엔드 중 기능이 가장 풍부한 Honcho를 중심으로 작동 방식을 살펴봅니다. Honcho는 Plastic Labs가 개발한 사용자 모델링 서비스로, Hermes 외에도 여러 에이전트 프레임워크와 연동됩니다.

- 변증법적 사용자 모델
- Honcho 도구
- 리콜 모드
- Honcho 서버 구성
- 설정

변증법적 사용자 모델

Honcho의 가장 큰 특징은 단순히 사실을 나열하는 것이 아니라, 사용자의 선호·패턴·의도를 **변증법적(dialectic)으로** 모델링한다는 점입니다. 예를 들어, 사용자가 평소에는 간결한 답변을 선호하지만 새로운 개념을 배울 때는 상세한 설명을 원한다는 미묘한 패턴까지 파악할 수 있습니다.

Honcho 서버의 deriver(추출기)가 대화를 분석하여 관찰을 자동으로 뽑아내고, 이 관찰들이 축적되면 서로 모순되는 패턴까지 고려한 종합적인 사용자 모델을 구축합니다.

Honcho 도구

Honcho가 활성화되면 5개의 전용 도구가 등록됩니다.

기능	설명
프로필 조회·갱신	피어 카드(이름, 역할, 선호, 패턴) 조회 또는 갱신
의미 검색	축적된 관찰에서 관련 정보를 검색
심층 추론	Honcho의 LLM에게 사용자에 대한 질문 (추론 깊이 5단계)
컨텍스트 조회	현재 세션의 전체 컨텍스트(요약, 관찰, 피어 카드, 최근 메시지) 확인
결론 저장	사용자에 대한 결론을 Honcho 메모리에 영구 저장 또는 삭제

심층 추론은 Honcho 서버의 LLM을 호출하여 종합적인 답변을 생성합니다. 단순 의미 검색보다 비용이 높지만, 축적된 관찰을 바탕으로 더 깊은 분석을 제공합니다.

리콜 모드

Honcho는 세 가지 리콜 모드를 지원합니다.

모드	자동 컨텍스트 주입	Honcho 도구 사용
`hybrid` (기본)	O	O
`context`	O	X
`tools`	X	O

기본값인 `hybrid` 모드에서는 매 턴 시작 시 Honcho의 컨텍스트가 자동으로 주입되고, 에이전트가 필요할 때 Honcho 도구를 추가로 호출할 수도 있습니다.

Honcho 서버 구성

Honcho는 클라우드 서비스(`app.honcho.dev`)를 이용하거나 직접 서버를 구축할 수 있습니다. 셀프 호스팅 시 Docker Compose로 4개 컨테이너를 실행합니다.

서비스	역할
api	REST API (FastAPI)
deriver	백그라운드 워커 — 관찰 추출, 요약, 임베딩
database	PostgreSQL + pgvector (벡터 검색)
redis	캐시

Honcho 서버는 자체적으로 LLM을 호출합니다. 클라이언트(Hermes)의 LLM을 빌려 쓰는 구조가 아니므로, 서버 측에도 별도의 LLM API 키가 필요합니다. 기본값은 OpenAI API(gpt-5.4-mini + text-embedding-3-small)이며, Hermes가 사용하는 LLM 설정과는 완전히 독립입니다.

> ⚠️ ***별도 비용 발생**: 클라우드 서비스를 이용하든 셀프 호스팅을 하든, Honcho가 사용하는 LLM 비용은 Hermes의 LLM 비용과 별개로 청구됩니다. 대화할 때마다 관찰 추출·요약·임베딩이 백그라운드에서 돌아가므로, 유료 API를 쓰면 그만큼 비용이 추가됩니다. 로컬 LLM(Ollama 등)을 연결하면 이 비용을 없앨 수 있습니다(부록 「로컬 LLM 가이드」의 Honcho 절 참고).*

설정

Hermes에서 Honcho를 활성화하는 방법은 간단합니다.

```
hermes memory setup honcho
```

설정 마법사에서 클라우드와 셀프 호스팅 중 선택하고, 접속 정보를 입력하면 됩니다. 또는 수동으로 지정할 수도 있습니다.

```
hermes config set memory.provider honcho
echo "HONCHO_API_KEY=발급받은-키" >> ~/.hermes/.env
```

API 키는 `app.honcho.dev`에서 발급하며, 셀프 호스팅 인스턴스를 쓸 경우 접속 URL을 지정합니다.

05-3-2. Mem0 — 간편 설정·사실 추출

Honcho가 기능이 가장 풍부한 백엔드라면, Mem0는 설정이 가장 단순한 백엔드입니다. 사용자의 사고방식·패턴까지 모델링하는 Honcho와 달리, Mem0는 대화에서 **사실(fact)을 추출·압축**하는 데 집중합니다. 과거 프로젝트의 세부 사항, 선호 설정, 반복 지시 같은 구체적인 정보를 정확히 기억하는 용도에 적합합니다.

- 작동 방식
- Mem0 도구
- 설정

작동 방식

대화가 끝나면 (사용자 메시지 + 에이전트 응답) 쌍이 Mem0 서버로 전송되고, 서버 측 LLM이 중요한 사실을 추출·압축하면서 중복을 제거합니다. 회수 시에는 의미 검색에 리랭킹(reranking)을 더해 관련도가 높은 기억을 골라냅니다. 추출·저장·중복 제거가 모두 서버에서 처리되므로, 에이전트가 무엇을 저장할지 직접 판단하지 않아도 됩니다.

Mem0 도구

Mem0가 활성화되면 3개의 도구가 등록됩니다.

도구	설명
`mem0_profile`	사용자에 대해 저장된 기억 전체 조회
`mem0_search`	의미 검색(필요 시 리랭킹)
`mem0_conclude`	LLM 추출을 거치지 않고 사실을 그대로 저장

> ⚠️ ***서버 측 LLM 비용**: Mem0도 사실 추출·중복 제거에 서버 측 LLM을 사용하는 클라우드 서비스입니다. 무료 티어를 넘어서면 사용량에 따라 Mem0 측에서 별도로 과금되며, 이는 Hermes가 쓰는 LLM 비용과 무관합니다.*

설정

API 키만 있으면 됩니다.

```
hermes memory setup    # 목록에서 mem0 선택
```

또는 수동으로 지정합니다.

```
hermes config set memory.provider mem0
echo "MEM0_API_KEY=발급받은-키" >> ~/.hermes/.env
```

API 키는 `app.mem0.ai`에서 발급합니다. `mem0ai` 패키지가 설치돼 있지 않으면 설정 과정에서 자동으로 설치됩니다.

세부 설정은 `$HERMES_HOME/mem0.json`에 둡니다.

키	기본값	설명
`user_id`	`hermes-user`	Mem0 상의 사용자 식별자
`agent_id`	`hermes`	에이전트 식별자
`rerank`	`true`	회수 정확도를 높이는 리랭킹 사용

05-4. 세션 검색

모든 대화 세션은 SQLite 데이터베이스(`~/.hermes/state.db`)에 저장됩니다. 에이전트에게 과거 대화를 찾아달라고 요청하면 전문 검색(Full-Text Search)으로 모든 세션을 검색합니다.

- 활용 시나리오
- FTS5 전문 검색
- CJK 검색을 위한 Trigram 인덱스
- 검색 범위
- 용량 관리

활용 시나리오

- "지난주에 Docker 네트워크 설정에 대해 이야기한 적 있지?"
- "이전에 그 API 에러를 어떻게 해결했더라?"
- "프로젝트 X에 대해 논의한 내용을 찾아줘"

MEMORY.md의 제한된 공간에 저장하지 못한 과거 정보도, 세션 검색을 통해 찾아낼 수 있습니다.

FTS5 전문 검색

검색은 SQLite의 FTS5(Full-Text Search 5) 엔진을 사용합니다. LLM을 호출하지 않고 인덱스에서 직접 결과를 가져오므로 빠릅니다.

FTS5는 고급 쿼리 문법을 지원합니다.

- 구문 검색: `"docker networking"` (정확한 구문)
- 불리언: `python NOT java`
- 접두사: `deploy*`
- OR 연산: `error OR warning`

검색 결과는 관련 대화(부모 세션)별로 그룹화되며, 현재 진행 중인 세션은 검색 대상에서 제외됩니다.

CJK 검색을 위한 Trigram 인덱스

기본 FTS5 토크나이저는 공백으로 단어를 구분합니다. 영어에는 잘 맞지만, 공백 없이 이어 쓰는 한·중·일 문자에서는 구문 검색이 제대로 작동하지 않습니다. 이를 보완하기 위해 3글자 단위로 겹쳐 인덱싱하는 **트라이그램(trigram) 인덱스**가 별도로 구성되어 있습니다. 한국어로 적은 부분 문자열도 검색됩니다.

인덱스	적합한 검색
기본 인덱스	영문 단어·구문
Trigram 인덱스	한국어·중국어·일본어 구문, 부분 문자열

에이전트가 검색할 때 어떤 인덱스를 쓸지 자동으로 판단하므로, 사용자가 구분할 필요는 없습니다.

> 💡 ***FTS5가 없는 환경**: 드물게 SQLite 빌드에 FTS5 모듈이 빠져 있으면(예: 일부 pip로 설치된 Python) 전문 검색이 비활성화되고 시작 시 경고가 표시됩니다. 이때 CJK 검색은 트라이그램 인덱스 대신 `LIKE` 검색으로 폴백합니다. 정식 설치 경로는 FTS5를 갖춘 Python을 함께 설치하므로 보통은 해당되지 않습니다.*

검색 범위

메시지 본문뿐 아니라 **도구 호출 내역**도 검색 범위에 포함됩니다. "지난주에 cronjob을 어떤 설정으로 만들었더라?" 같은 질문도 검색으로 찾을 수 있습니다.

용량 관리

세션 기록은 계속 쌓이며, 기본 설정에서는 자동 정리가 꺼져 있습니다. 가벼운 사용이라면 수백 세션에 10~15MB 수준이지만, 게이트웨이나 크론을 많이 쓰면 수백 MB까지 커질 수 있습니다.

자동 정리를 켜면 종료된 세션 중 보관 기간이 지난 것을 시작 시 삭제합니다.

```
# ~/.hermes/config.yaml
sessions:
  auto_prune: true        # 자동 정리 (기본값: false)
  retention_days: 90       # 종료된 세션 보관 기간
```

진행 중인 세션은 삭제되지 않습니다. 장기 보존이 필요한 세션은 정리를 켜기 전에 `hermes sessions export`로 내보낼 수 있습니다.

오래 사용한 저장소는 FTS5 인덱스 세그먼트가 늘어나 검색이 느려지거나 디스크를 더 차지할 수 있습니다. 이때 `hermes sessions optimize`를 실행하면 인덱스 세그먼트를 병합하고 데이터베이스를 VACUUM해서 검색 성능을 회복하고 용량을 회수합니다. 세션 데이터 자체는 그대로 유지됩니다.

05-5. 컨텍스트 압축

대화가 길어지면 LLM의 컨텍스트 윈도우가 가득 찹니다. Hermes Agent는 이 문제를 자동으로 해결합니다. 사용자가 개입할 필요 없이, 토큰 사용량이 임계값에 도달하면 대화 중간 부분을 요약해서 공간을 확보합니다.

- 작동 방식
- 압축 트리거
- 메모리와의 관계
- 설정
- 수동 명령
- 보조 모델 실패 처리

작동 방식

```
flowchart TD
    T["토큰 ≥ 임계값\n또는 413/컨텍스트 오류"] ⟶ P1["1단계: 도구 출력 가지치기\n(LLM 호출 없음)"]
    P1 ⟶ P2["2단계: 경계 보호\n앞 3개 + 뒤 20개 메시지 보존"]
    P2 ⟶ AUX{"보조 모델\n사용 가능?"}
    AUX -- "예" --> P3["3단계: 중간 턴을\n보조 모델로 요약"]
    AUX -- "아니오" --> DROP["요약 없이\n중간 턴 삭제 ⚠️"]
    P3 ⟶ OK{"요약\n성공?"}
    OK -- "예" --> DONE["요약으로 중간 턴 대체\n압축 완료"]
    OK -- "아니오" --> RETRY["메인 모델로\n한 번 재시도"]
    RETRY ⟶ OK2{"재시도\n성공?"}
    OK2 -- "예" --> DONE
    OK2 -- "아니오" --> DROP
```

압축은 세 단계로 진행됩니다.

1단계 — 도구 출력 가지치기 (LLM 호출 없음)

오래된 도구 실행 결과 중 200자를 넘는 것을 짧은 플레이스홀더로 교체합니다. LLM 호출 없이 빠르게 토큰을 줄이는 사전 작업입니다.

2단계 — 경계 보호

대화의 처음과 끝을 보호합니다.

- **앞쪽 3개 메시지** — 시스템 프롬프트와 첫 대화를 유지합니다.
- **뒤쪽 최근 메시지** — 기본값 20개 메시지(`protect_last_n`)를 압축 대상에서 제외합니다. 최근 작업 맥락이 유지됩니다.

3단계 — LLM 요약

보호 구간 사이의 중간 턴을 보조 모델(저렴한 모델)에게 보내 구조화된 요약을 생성합니다.

```
## Goal
[사용자가 달성하려는 목표]

## Progress
### Done / In Progress / Blocked

## Key Decisions
[중요한 기술 결정과 이유]

## Relevant Files
[읽거나 수정한 파일 목록]

## Next Steps
[다음에 해야 할 작업]
```

재압축 시에는 이전 요약을 처음부터 다시 만들지 않고 **누적 업데이트**합니다. 압축을 여러 번 거쳐도 초반 맥락이 점진적으로 사라지지 않습니다.

압축 트리거

압축이 시작되는 시점은 세 가지입니다.

트리거	시점	작동
사전 압축	API 호출 직전, 추정 토큰 ≥ 임계값	최대 3회 반복 압축
413 Payload Too Large	API가 413 응답 반환 시	압축 후 2초 대기, 재시도
컨텍스트 길이 오류	"context length exceeded" 등 감지 시	컨텍스트 한도를 하향 조정 후 압축

세 번째 트리거는 로컬 백엔드(Ollama, LM Studio 등)에서 특히 중요합니다. 모델마다 실제 컨텍스트 한도가 다르고, 에러 메시지 형식도 제각각이기 때문입니다. Hermes Agent는 에러 메시지에서 실제 한도를 파싱하거나, 단계적으로 한도를 낮추며

재시도합니다.

메모리와의 관계

컨텍스트 압축은 **메모리 도구를 호출하지 않습니다**. 압축 과정에서 MEMORY.md나 USER.md를 읽거나 쓰는 일은 없으며, 요약 생성도 도구 없이 LLM 텍스트 생성만으로 처리됩니다.

요약기는 메모리 파일에 직접 접근하지 않지만, 요약 내용이 기존 메모리와 충돌하지 않도록 설계되어 있습니다.

설정

```
# ~/.hermes/config.yaml
compression:
  enabled: true            # 기본값: true
  threshold: 0.50          # 모델 컨텍스트 한도의 50%에서 압축 시작
  target_ratio: 0.20       # threshold 토큰 예산의 20%를 최근 대화로 보존
  protect_last_n: 20       # 최근 20개 메시지는 압축에서 제외
  summary_model: null      # 요약용 모델 (미지정 시 보조 모델 자동 선택)
```

`target_ratio`는 `threshold` 토큰 예산에 대한 비율입니다. 기본값(threshold 0.50 × target_ratio 0.20)에서는 컨텍스트의 약 10%를 최근 대화로 보존합니다. 200K 모델이면 약 20K, 1M 모델이면 약 100K입니다.

`threshold`를 낮추면 더 일찍 압축이 시작되어 요약 품질이 높아지지만, 압축 횟수가 늘어납니다. 로컬 모델처럼 컨텍스트가 작은 환경에서는 `0.40` 정도로 낮추는 것이 안정적입니다.

수동 명령

자동 압축 외에 수동으로도 제어할 수 있습니다.

명령어	설명
`/compress`	즉시 압축 실행
`/compress <focus>`	주제 지정 압축. 예: `/compress "API 설계 결정"`
`/compress here [N]`	경계 인식형 부분 압축. 최근 N개 교환은 그대로 두고 그 이전만 요약 (N 생략 시 2)
`/usage`	현재 토큰 사용량과 임계값 확인

`/compress <focus>` 형식은 요약기에 "이 주제 중심으로 정리해 줘"라는 힌트를 줍니다. 긴 토론 끝에 결정 사항만 남기고 부수적인 시행착오는 줄이고 싶을 때 유용합니다.

`/compress here [N]` 형식은 압축 경계를 사용자가 직접 지정합니다. "여기까지는 요약하고 최근 대화는 그대로 두자"라는 뜻으로, 최근 N개 교환(사용자 메시지와 그에 딸린 응답 묶음)을 원문 그대로 보존합니다. 방금 나눈 대화는 손대지 않으면서 그 이전의 긴 맥락만 정리하고 싶을 때 유용합니다.

> 💡 *대화가 느려지거나 응답이 반복되는 느낌이 들면 `/usage` 로 토큰 사용량을 확인해 보세요. 임계값에 가까워졌다면 `/compress` 로 미리 정리하는 것도 방법입니다.*

보조 모델 실패 처리

압축은 보조 모델을 사용합니다. 보조 모델이 오류를 내면, 포기하기 전에 **메인 모델로 한 번 더 재시도**합니다. 그래도 실패하면 사용자에게 그 사실이 표시되고, 압축 없이 호출이 진행됩니다. 메인 모델로 자동 전환(폴백)이 작동하더라도 보조 모델 설정에 문제가 있다면 `hermes status` 나 로그에서 확인할 수 있습니다.

보조 모델 자체가 설정되어 있지 않으면, 세션 시작 시 "⚠ No auxiliary LLM provider configured" 경고가 표시됩니다. 이 상태에서 압축이 발동하면 **요약 없이 중간 턴을 그냥 삭제**합니다. 대화 맥락이 크게 손실될 수 있으므로, `hermes setup` 을 실행하거나 `OPENROUTER_API_KEY` 환경변수를 설정해 보조 모델을 연결하는 것을 권장합니다.